Multiband image classification with a distributed architecture

Ian J Curington and Stephen E Cannon

Much research has gone into specialized hardware for remotely sensed imagery analysis applications, particularly in the use of Landsat data for feature classification analysis. The paper outlines a particular multiband classifier and shows expected performance using the FPS-5000 array processor. The advantage of distributed resources are shown in an optimized implementation of the algorithm in a particular processing environment.

Keywords: remote sensing, feature classification, Landsat

Pattern classification is a common task in many image analysis applications. This is particularly true in the use of Landsat data where classification is used to separate specific feature characteristics such as crop type or mineral content. This paper outlines a multiband classifier and shows expected performances for several image sizes, numbers of bands, and numbers of classes. The advantages of distributed resources are shown in an optimized implementation of the algorithm, using an FPS-5000 array processor.

Multiband imagery classification

Image classification is a two-step process[1]. First, the classes of interest (corn, wheat, pavement etc.) are characterized through the analysis of data samples which are chosen as representative of a particular class. At this stage, the classifier is 'trained'. Second, all remaining image elements (pixels) are classified by rules which utilize the class characteristics. This methodology is oriented towards computer implementation, permits rapid and repeatable analysis, and is easily tailored to a wide range of classification problems. The technique is most applicable when the goal is to categorize each image element into a limited number of discrete classes. Thus possible uses of the method include classification of farm land into specific crop species, and exposed ground into specific mineral content classes.

A multiband image is represented in digital form by a set of values at each spatial position (Figure 1) describing measured or estimated values of spectral energy (red, green, blue, infrared etc.), spatial gradient, temporal history, or other analytic measures. The combined values for a given image point can be thought of as a vector in feature or pattern space.

The basic premise of classification techniques is that the pixel vectors representing a particular class form a compact region in pattern space (see Figure 2). If the pixel vectors are randomly distributed, no distinct class can be found. Most images from satellites have well defined classes, since most ground objects reflect unique spectra of energy according to their type.

If the location and characteristics of clusters in the pattern space can be determined and identified with

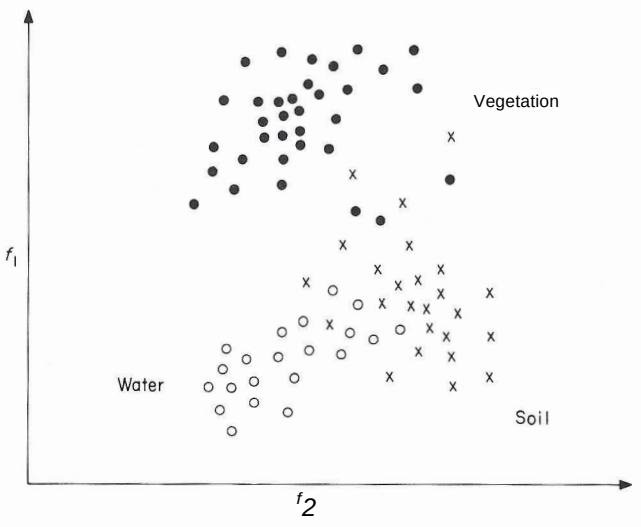

Figure 7. Clusters in pattern space

Floating Point Systems Inc., PO Box 23489. Portland, OR 97223, USA

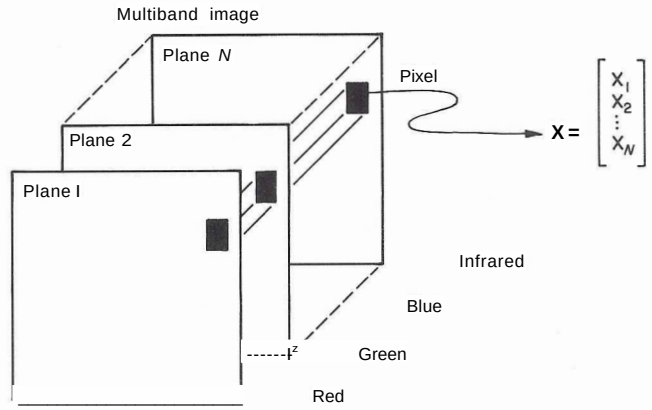

Figure 2. A pixel in vector notation

specific classes, a decision rule can be developed to determine which other pixels fall within these classes. The process of selecting classes of interest can be performed automatically: however, it is most often done interactively, using operator knowledge of the scene. This is referred to as building the 'training set' of class descriptions, and is performed by selecting representative samples of a certain ground cover. Once a training set exists, it can be used by the classifier as described below, to classify an entire multiband image. Some applications require as few as two classes, others up to 30. Broad classes are used initially, and refined into subclasses later.

Distributed processor architecture

Better performance of this and similar algorithms is obtained by using a multiple-instruction multiple-data-path approach, than with traditional single-instruction methods[2]. The FPS-5000 Series array processor uses a control processor which, executing a single instruction stream, is rated at 12M floating point operations per second (FLOPS). The control processor is assisted by multiple XP32 computational elements (each rated at 18M FLOPS). Because the XP32s are separate array processor elements, sharing a portion of the processing load, they can operate concurrently: thus a fully configured FPS-5000 with three XP32s, for example, is rated at 62M FLOPS.

The FPS-5000 control processor has a parallel floating point multiplier and adder, and data, program and coefficient memories. The XP32 coprocessor has one floating point multiplier, two floating point adders, writeable control store, local high-speed data memory, and coefficient memories, with 32-bit format. Both processors are able to operate in parallel, while data movement is controlled by separate DMA controllers.

Multiband Mahalanobis classifier

As an example consider the design and implementation of a Mahalanobis classifier[3] as applied to multiband images. This classifier can be used on any size of image, numberof bands, and numberof classes. Typical image sizes range from 512x512 pixels to 6144 x 6144, from two to 12 bands (pixel vector lengths), and from two to 16 classes[4]. The Mahalanobis classifier represents a tradeoff between adequate class discrimination and computational efficiency. Alternative classifiers can be chosen, such as a parallelepiped classifier (for fast execution at the expense of accuracy) or a maximum-likelihood classifier (for more accurate class distinctions but with higher computational costs). Practice has shown the Mahalanobis classifier to be robust[3], ie able to give reasonable results when class statistics violate input assumptions.

General algorithm

The algorithm operates on each pixel, and the following discussion considers the operations on one pixel vector. The pixel is input to the classifier, which computes a distance measure to each class. Once all distance measures have been evaluated, the minimum is chosen as a decision rule, the output being a class association tag. This tag can then be used for further analysis or to false-colour a composite image for interactive viewing.

Derivation of training classes

The general technique for finding the training set for a given class varies among user applications. For each class, a group of samples is collected and analysed. If the number of bands is *N,* a pixel **X** is composed of $\mathbf{X}_1, \mathbf{X}_2 \ldots \mathbf{X}_N$. This is normally written as

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_N \end{bmatrix} \quad (1)$$

After *K* pixels have been selected as representative of a class, class characteristics are computed. By gaussian statistics, it is sufficient to estimate the mean and covariance to characterize completely the class statistics[5].

The mean vector **IVI** of the class J is estimated as

$$\mathbf{M}_J = \frac{1}{K} \sum_{i=1}^{K} \mathbf{X}_{iJ} = \frac{1}{K} \sum_{i=1}^{X} \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_N \end{bmatrix}_J \quad (2)$$

where Mj is a vector of length *N.* X_{zJ} is the set of pixels selected to belong to the clan J (with / the index of each pixel in this group of size *K),* and the sum is a vector sum. This yields the statistical 'centre' of the class.

A measure of the size and A/-dimensional shape of the class is given by the covariance matrix C. This is a symmetric positive-definite A/x/V matrix. The training set samples and the mean vector are used to estimate the covariance matrix according to

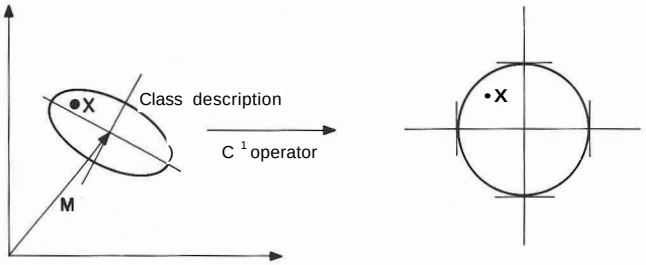

Figure 3. Mahalanobis distance from one class to a pixel (the vertical and horizontal axes represent each of the two feature variables: X is a class member: the class is the interior of the ellipse and circle')

$$C_1 = \frac{1}{A-1} \sum_{j=1} (X_j - M_J)(X_j - M_J)^T \qquad (3)$$

For each class, a mean vector (M,) and covariance matrix (C,) are computed, representing that area in feature space where the class is clustered. Figure 3 shows Mahalanobis classification for one class with two feature variables.

Discriminant function

Once class characteristics have been obtained, they are used by a discriminant function to estimate the distance in feature space from an unknown pixel to a given class (see Figure 3). With the use of multiple computational elements, these discriminant functions can operate in parallel, each estimating a distance to its particular class. Finally, a decision rule is applied to determine the best match.

Many distance measures can be applied. This example uses the Mahalanobis distance (a scalar distance) of a pixel with respect to a class (modelled by **M** and C). Because the distance measure represents a large portion of the processing time for a classifier, it should be chosen carefully.

The Mahalanobis discriminant function is given by

$$ty = (Z-M)^r C^{-1} (Z-M) \qquad (4)$$

where Z is an arbitrary vector, and C represents the inverse of C as shown in equation (3) above. M is given by equation (2). The implied dimension of the operands M, C and Z is *N; d* is the scalar distance measure.

The basicform of the measure is a dot product, and is written as a double sum with all vector and matrix subscripts shown as

$$d = \sum_{/=i} \sum_{/=i} m W \qquad (5)$$

where

$$T, = Z, - M \qquad (6)$$

Implementation design

Although the implementation is in a specialized writeable control store instruction set, the algorithm is more easily understood if shown in a high-level language.

The application of a Mahalanobis discriminant function is illustrated by a nested loop in FORTRAN as

```
   DO 3 K=1, NCLASS
                        /* loop through all classes */
   DO 1 1=1, N
1  T(I)=X(I)—MEAN (K, I)
          /* assign difference to temporary store */

   D=0.
   DO 2 1=1, N              /* discriminant loop */
   DO 2 L=1, N
2  D=D+ T(I)*T(L)*CINV(K, I, L)
3  DIST(K) = D

   TDIST=VLARGE
   DO 4 K=1, NCLASS
          /* find minimum class distance rule */
   IF (DIST(K).GT.TDIST) GOTO 4
   TDIST  DIST(K)
   NEWK=K       /* NEWK is final class for pixel */
4  CONTINUE
```

Not shown in this per-pixel process is the mechanism to read groups of pixels, perhaps scan lines, and to buffer and output class assignments for every pixel vector. The outer loop shown above evaluates each class sequentially, saving measures in the DIST array. Once finished, the classifier rule of minimum *d* is applied in a single pass. The method shown above normally occupies one routine, and would be called for each pixel. An alternative organization allows pipelining and vector operations for an array processor.

This organization could be pipelined in several ways. Each processor could evaluate the above code for a different group of pixels. Alternatively, all processors could evaluate the same pixels against different groups, with the classifier pass reformed afterwards, as shown in Figure 4.

The above algorithm, reorganized so that passes over the data (many pixels) are at the innermost loop position (at label 3), is written

```
   DO 3 K=1, NCLASS
   DO 1 1=1, N
   DO 1 KK=1, M
1  T(KK, I)=X(KK, I)—MEAN (K,I)

   DO 2 1=1, M
```

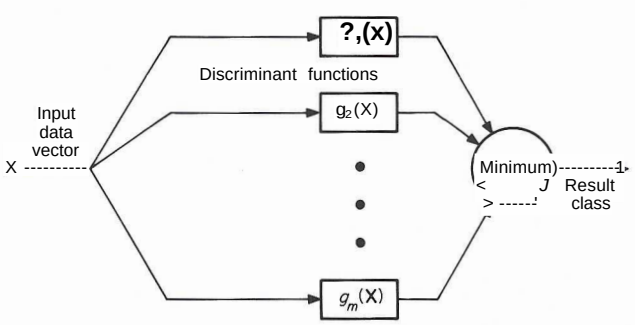

Figure 4. A pattern classifier is defined in terms of discriminant functions

```
2  DIST (I)=0.0

   DO 3 L=1, N
   DO 3 1=1, N
   DO 3 KK=1, M
3  DIST(KK) = DIST(KK)
                + T(KK, I)*T(KK, L)*CINV(K, I, L)
```

where *K* iterates over classes, *M* is the number of pixels to process at once (scan line length), *KK* iterates over the array of pixels, and A/is the dimension of each pixel. This organization allows efficient data flowthrough the pipelines of the XP32.

Allocation of hardware to implement classifier

Figure 5 shows the flow of data through the array processor's coprocessor system. The host or some externally attached large image memory buffers the original image. A certain number of scan lines are brought into the main data memory and given to one or more XP32 coprocessors, where the discriminant function is evaluated. The discriminant values are transferred back into the main data memory where the control processor performs the decision rule processing, as in Figure 6. The results are moved to external image memory for later analysis or display.

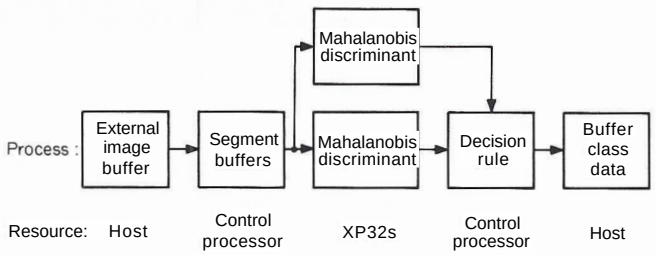

Figure 5. Data flow organization

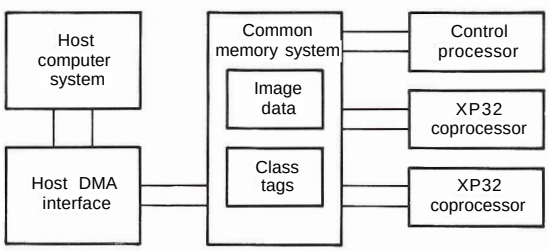

Figure 6. Multiprocessor organization

Performance estimation

Although the XP32 is not user programmable, the following analysis reflects internal timing and algorithmic structure. If the discriminant function is placed in XP32 microcode, the architecture allows the following loop organization

1 fetch DIST
2 fetch T(i) and T(j)
3 compute (two multiplies, one add)
4 store new DIST

Since the XP32 is able to perform two-memory references and one multiply per cycle, the loop is multiplier limited.

Overlapping memory reference and ALU operations yield an effective two-cycle loop. The mean subtraction loop can be one cycle, since fetch and store are overlapped with a single subtract.

Given a 6 MHz clock cycle of the XP32, the time to find the discriminant values for each class for one image scan line using one XP32 coprocessor is

[(0.333 /zs)/VA/W+ (0.167 jrs)/V/W]/VCLASS+ loop overhead

This performance estimate assumes that the covariance and mean operators are precomputed and stored in internal registers during the inner loops. In addition the loops are constructed in microcode such that the pipes are full, and all data structures are prearranged in the XP32 memory. The overhead of transferring data to and from the XP32 is assumed to be 100% overlapped with processing. This time does not include the final pass to determine class assignment.

A summary of performances for various processors is shown in Table 1.

Overhead is arbitrarily set to 10% ; this approximates the setup and control overhead of the XP32, and the time needed for communication and synchronization

Table 1. Performance summary for various processors

			Time, ms			
Scanline length (A4)	Number of bands (A/)	Classes (A/CLASS)	FPS-5110	FPS-5420	FPS-100	VAX 11/780*
512	4	2	6.7	3.4	18.5	253
512	8	4	51	26	145	1790
512	12	4	113	57	325	3970
512	4	16	54	27	149	1890
3240	4	8	171	86	470	6320
6144	4	12	486	243	1338	19260
6144	7	8	946	473		
8192	7	32	5042	2521		

*With Floating Point accelerator

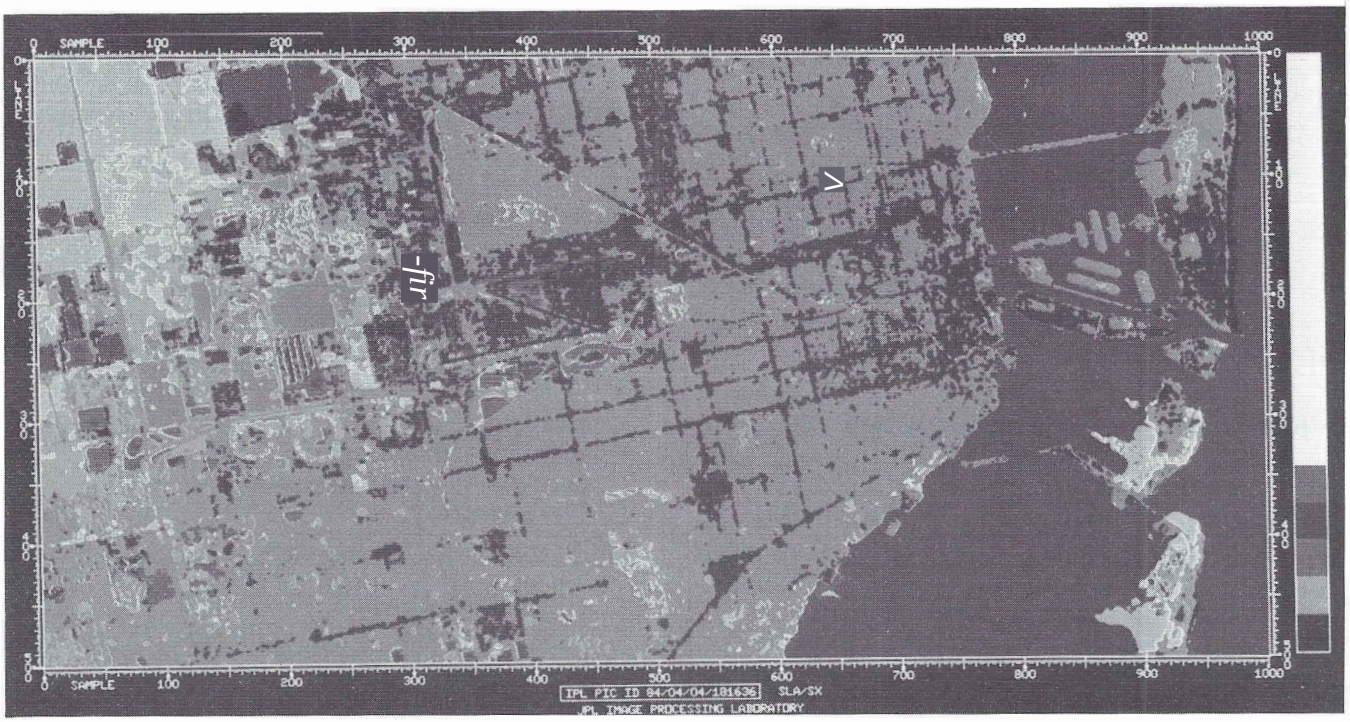

Figure 7. Image created using the image processing system VICAR/IBIS (video image communication and retrieval/image-based information system} developed at the California Institute of Technology Jet Propulsion Laboratory. It represents the results of a mu/tispectral unsupervised classification algorithm. Unsupervised statistics were gathered from all seven bands of the thematic mapper for 17 USGS level II land use categories. A classification image was then produced using VICAR program FASTCLAS, which uses the FPS array processor. Accuracy measurements were then computed using various IBIS programs. A low-pass box filter was applied to the image to remove any high-frequency noise. Finally, a pseudocolour lookup table corresponding to the manually digitized land use file was applied to the classification image

between the processors. Whole image times are not simply the above times multiplied by a number of scan lines, since the image must be buffered externally to the array processor.

If more than one coprocessor is available, these times can be reduced, ie halved for two XP32s, as long as system memory bandwidth is not surpassed. The final classification pass performed by the main array processor is fast compared with the times in Table 1 and can be overlapped. If an efficient external buffering scheme is available using the host or external bulk storage, the time for full images can be estimated by multiplying the scan line times by the number of lines.

Summary

High-performance implementations of multiband classification can be achieved by assigning multiple hardware processors such as the XP32 coprocessors in the FPS-5000 array processor (eg see Figure?). By taking advantage of algorithmic symmetry, multiple coprocessors execute in parallel and can dramatically reduce the processing time. In such a processing environment, common classification functions such as by Mahalanobis are excellent candidates for conversion.

Acknowledgements

The authors are indebted to Steven Adams of the Earth Resources Applications Group of the Jet Propulsion Laboratory for supplying implementation experience and illustrations.

References

1 **Swain and Davis** *Remote sensing: the quantitative approach* McGraw-Hill, New York, USA (1978) pp 137-185

2 **Tracy, R** *Distributed system architectures for high throughput array processors* Floating Point Systems, USA (1983)

3 **Moik, Johannes G** 'Digital processing of remotely sensed images' *Rep. SP-431* Goddard Space Flight Center, NASA, USA (1980)

4 **Bradford and Lewis** 'Applications processing systems for imagery data' *Array User Group Conf.* Floating Point Systems, USA (1978)

5 **Mood, Graybill and Boes** *Introduction to the theory of statistics* McGraw-Hill, New York, USA 3rd edn (1974)

Bibliography

Mori and Kidode 'Design of local parallel pattern processorfor image processing' *AFIPS Conf. Proc.* Vol. 47 (1978) pp 1025-1031